

Mapping physicochemical features onto 2D interface planes for protein–protein binding affinity prediction

Report di progetto – Documentazione tecnica della pipeline (Task 1–4)

Advanced Machine Learning for Physics

September 16, 2026

Abstract

Questo report descrive il lavoro svolto nell’ambito del progetto proposto da E. Milanetti e G. Ruocco per il corso *Advanced Machine Learning for Physics*, finalizzato alla predizione dell’affinità di legame proteina–proteina a partire da mappe 2D di complementarità costruite tramite espansione in polinomi di Zernike. Il documento copre la Task 1 del progetto (identificazione dell’interfaccia e estrazione delle patch di superficie), descrivendo dataset, pipeline implementata, scelte metodologiche e un esempio rappresentativo di output.

Contents

1	Introduzione	2
2	Dataset	2
3	Architettura della pipeline (Task 1)	2
3.1	Lettura del dataset	3
3.2	Parsing delle strutture e download	3
3.3	Identificazione dell’interfaccia	3
3.4	Estrazione delle patch e generazione della superficie	3
3.5	Orchestrazione e output	4
4	Post-elaborazione: selezione dei dimeri e pulizia degli output	4
4.1	Selezione dei dimeri (<code>filter_dimers.py</code>)	5
4.2	Pulizia delle cartelle (<code>clean_task1.py</code>)	5
5	Risultati dell’esecuzione completa	5
5.1	Esempio: complesso 1KTZ	5
5.2	Statistiche aggregate sui complessi dimerici	6
6	Task 2 – Mappatura della complementarità tramite descrittori di Zernike	6
6.1	Nota metodologica: natura locale dei descrittori	7
6.2	Algoritmo implementato	7
6.3	Esecuzione su HPC	8
6.4	Punti aperti	8
7	Task 3 – Costruzione dei piani di complementarità 2D	8
7.1	Proiezione PCA sul piano medio di interfaccia	9
7.2	Filtro energetico di Lennard-Jones	9
7.3	Discretizzazione su griglia 32×32 e canali	9
7.4	Standardizzazione Z-score globale	9

8 Task 4 – Predizione dell'affinità tramite CNN	10
8.1 Target	10
8.2 Architettura InterfaceCNN	10
8.3 Protocollo di addestramento	10
8.4 Risultati	10
9 Discussione e limiti	11
10 Sviluppi futuri	11

1 Introduzione

L'obiettivo generale del progetto è costruire un framework fisicamente motivato per la predizione dell'affinità di legame (K_D o ΔG) tra coppie di proteine, rappresentando l'interfaccia di legame come una mappa 2D multi-canale (complementarità geometrica, potenziale elettrostatico, idrofobicità) da utilizzare come input per una rete neurale convoluzionale (CNN). Il progetto è strutturato in cinque task successive; questo report documenta la **Task 1**, propedeutica a tutte le fasi successive, che consiste nell'identificazione delle interfacce di legame a partire da un dataset di complessi proteina-proteina e nell'estrazione delle rispettive patch di superficie molecolare.

2 Dataset

Il dataset di partenza è l'*Affinity Benchmark v5.5*, fornito in formato Excel/CSV, contenente per ciascun complesso proteina-proteina:

- l'identificativo PDB della struttura cristallografica;
- le catene appartenenti al *ligand* e al *receptor*;
- il nome delle due proteine coinvolte;
- la costante di dissociazione K_D (in M);
- metadati sperimentali (metodo di risoluzione strutturale, risoluzione in Å, temperatura, riferimenti bibliografici/PubMed).

Il file utilizzato conta 209 complessi. A titolo di esempio, la prima riga del dataset descrive il complesso 1KTZ (TGF-beta / TGF-beta receptor, catena ligando A, catena recettore B, $K_D = 2.9 \times 10^{-7}$ M, risolto per diffrazione a raggi X a 2.15 Å).

3 Architettura della pipeline (Task 1)

Il codice della Task 1 è organizzato come package Python con responsabilità separate per ciascuna fase della pipeline:

- `dataset.py` – lettura del dataset e costruzione dei record dei complessi;
- `pdb.py` – parsing/scrittura di file PDB e download automatico da RCSB;
- `interface.py` – identificazione dei residui di interfaccia tramite distanze atomiche;
- `surface.py` – generazione della superficie molecolare "sampler" con densità elettronica approssimata;

- `chimerax.py` – generazione della superficie molecolare tramite ChimeraX (superficie escludente il solvente) e suo filtraggio sui residui di interfaccia;
- `pipeline.py` – orchestrazione dell'intero flusso per ciascun record del dataset;
- `cli.py` – interfaccia a riga di comando per l'esecuzione della pipeline.

Il flusso complessivo, per ciascun complesso del dataset, è il seguente:

1. lettura del record (PDB ID, catene ligando/recettore, K_D) da `dataset.py`;
2. recupero della struttura 3D (file locale o download da RCSB) e parsing degli atomi;
3. separazione degli atomi in catene ligando e catene recettore;
4. identificazione dei residui di interfaccia tramite cutoff di distanza atomica;
5. estrazione degli atomi appartenenti ai residui di interfaccia (patch);
6. generazione della superficie molecolare e annotazione con densità elettronica approssimata;
7. scrittura degli output (strutture, residui di interfaccia, superfici, metadati) per il complesso.

3.1 Lettura del dataset

Il modulo `dataset.py` legge il file Excel del benchmark e costruisce, per ogni riga valida, un oggetto `ComplexRecord` contenente PDB ID, catene di ligando e recettore (con parsing robusto di separatori multipli come virgole, punti e virgola o spazi), K_D e metadati testuali. Sono previsti filtri opzionali per limitare l'elaborazione a un sottoinsieme di PDB ID, a un intervallo di righe, o ai soli complessi *dimerici* (una sola catena per lato).

3.2 Parsing delle strutture e download

Il modulo `pdb.py` implementa un parser leggero del formato PDB a colonne fisse, che estrae per ciascun atomo posizione, elemento chimico, catena, numero e nome di residuo, scartando le molecole d'acqua e gestendo gli *altloc*. Le strutture mancanti localmente vengono scaricate automaticamente da RCSB (<https://files.rcsb.org>). Sono inoltre presenti funzioni di selezione delle catene di interesse e di scrittura di sottoinsiemi di atomi in nuovi file PDB, utilizzate per isolare ligando, recettore e le rispettive patch di interfaccia.

3.3 Identificazione dell'interfaccia

L'interfaccia di legame viene identificata in `interface.py` calcolando, per ogni coppia di atomi pesanti (ligando, recettore), la distanza euclidea e trattenendo i contatti con distanza inferiore a un cutoff (di default **5.0 Å**). Il calcolo è vettorizzato tramite `numpy`, processando gli atomi del ligando a blocchi (*chunk*) per limitare l'uso di memoria sulle matrici di distanza. Per ciascuna coppia di residui in contatto vengono registrati la distanza minima e il numero di contatti atomici, mentre l'insieme complessivo dei residui di interfaccia (lato ligando e lato recettore) viene restituito come due insiemi separati.

3.4 Estrazione delle patch e generazione della superficie

Una volta identificati i residui di interfaccia, gli atomi corrispondenti vengono estratti (`atoms_in_residues`) e scritti in file PDB dedicati (catene complete e patch di interfaccia, separatamente per ligando e recettore).

La superficie molecolare può essere generata con due backend alternativi, selezionabili da riga di comando:

Sampler (`surface.py`) – per ogni atomo della patch viene generato un set deterministico di punti sulla sfera (distribuzione di Fibonacci, di default 64 campioni per atomo) a distanza raggio di van der Waals + raggio della sonda (probe radius, default 1.4 Å, valore tipico dell'acqua). I punti "sepolti" da altri atomi dello stesso partner vengono scartati tramite un test di occlusione basato sui raggi atomici. I punti rimanenti sono annotati con una densità elettronica approssimata, calcolata come somma di contributi gaussiani pesati per numero di elettroni degli atomi vicini entro un raggio di cutoff (default 6 Å), con decadimento controllato da un parametro α (default 1.8).

ChimeraX (`chimerax.py`) – la superficie viene generata invocando ChimeraX in modalità *headless* (`-nogui`) sul comando **surface**, che calcola una superficie escludente il solvente (*solvent-excluded surface*) sull'intera catena, esportata in formato OBJ. I vertici OBJ vengono poi associati al residuo più vicino e annotati con la stessa densità elettronica approssimata usata nel backend sampler. La strategia adottata di default (`full_chain_then_filter`) calcola la superficie sull'intera catena e la filtra successivamente sui soli residui di interfaccia, per evitare artefatti di bordo che si avrebbero calcolando la superficie direttamente sulla patch isolata; è comunque disponibile una modalità "legacy" che opera direttamente sulla patch.

Il backend di default nella pipeline è ChimeraX con filtraggio "full-chain-then-filter".

3.5 Orchestrazione e output

Il modulo `pipeline.py` coordina l'intero processo per ciascun record del dataset, gestendo anche il fallimento silenzioso di singoli complessi (ad es. struttura mancante o catene non trovate), che vengono registrati come errori senza interrompere l'elaborazione del batch. Per ogni complesso elaborato con successo viene creata una sottocartella `outputs/task1/{indice}_{PDB_ID}/` contenente:

- `ligand_chains.pdb`, `receptor_chains.pdb` – catene complete;
- `ligand_interface_patch.pdb`, `receptor_interface_patch.pdb` – patch di interfaccia;
- `ligand_interface_residues.csv`, `receptor_interface_residues.csv` – elenco dei residui di interfaccia (catena, numero, codice di inserzione, nome residuo);
- `interface_contacts.csv` – contatti residuo–residuo con distanza minima e numero di contatti atomici;
- `ligand_surface_patch.csv`, `receptor_surface_patch.csv` – superficie molecolare annotata (coordinate, normali, densità, residuo di appartenenza);
- `metadata.json` – parametri usati, conteggi di atomi/residui/punti di superficie ed esito dell'elaborazione.

Al termine dell'esecuzione viene inoltre prodotto un `manifest.csv` riassuntivo con lo stato (*ok/error*) di ciascun complesso processato. L'intera pipeline è esposta tramite CLI (`cli.py`), con parametri configurabili per cutoff di interfaccia, backend di superficie, numero di campioni per atomo, raggio della sonda e parametri del modello di densità.

4 Post-elaborazione: selezione dei dimeri e pulizia degli output

Poiché il progetto si concentra su interfacce a due sole catene (un ligando e un recettore), l'insieme dei complessi processati dalla pipeline è stato in seguito ristretto ai soli casi **dimerici** tramite due script di post-elaborazione, eseguiti successivamente alla Task 1 vera e propria.

4.1 Selezione dei dimeri (`filter_dimers.py`)

Lo script legge nuovamente il file Excel del benchmark e individua i complessi dimerici imponendo che sia la colonna **Ligand Chains** sia la colonna **Receptor Chains** contengano un singolo carattere (una sola catena per lato, senza elenchi separati da virgole). Per ciascun PDB ID identificato come dimero, la cartella corrispondente viene cercata in `outputs/all` (l'output completo della pipeline su tutto il dataset) e copiata in `outputs/task1`, che diventa così la directory di lavoro contenente esclusivamente i complessi dimerici.

4.2 Pulizia delle cartelle (`clean_task1.py`)

Una volta popolata `outputs/task1` con i soli dimeri, un secondo script rimuove da ciascuna sottocartella tutti i file non essenziali alle fasi successive del progetto, mantenendo esclusivamente:

- `ligand_surface_patch.csv`, `receptor_surface_patch.csv` – superfici di interfaccia annotate;
- `ligand_interface_residues.csv`, `receptor_interface_residues.csv` – elenco dei residui di interfaccia;
- `metadata.json` – parametri e conteggi dell'elaborazione.

File intermedi come le strutture PDB delle catene complete, le patch in formato PDB, i file OBJ di ChimeraX e i contatti residuo-residuo vengono eliminati, riducendo lo spazio occupato e mantenendo in `outputs/task1` solo gli artefatti necessari come input per la Task 2 (mappatura Zernike).

5 Risultati dell'esecuzione completa

Il dataset di partenza (207 complessi) è stato elaborato interamente dalla pipeline di Task 1, ma il progetto si concentra esclusivamente sui **complessi dimerici** (una sola catena per ligando e una per recettore), selezionati con lo script `filter_dimers.py` descritto in Sezione 4: applicando questo criterio si ottengono **120 complessi dimerici**, tutti elaborati con successo dalla pipeline (nessun errore di download, parsing o calcolo della superficie). Le statistiche riportate di seguito si riferiscono esclusivamente a questo sottoinsieme.

5.1 Esempio: complesso 1KTZ

A titolo illustrativo, la Tabella 1 riporta i conteggi prodotti dalla pipeline per il complesso 1KTZ (TGF-beta / TGF-beta receptor, catena ligando A, catena recettore B) – uno dei dimeri del sottoinsieme – con cutoff di interfaccia a 5.0 Å e backend ChimeraX (`full_chain_then_filter`).

I residui di interfaccia identificati sul lato ligando includono, tra gli altri, ARG25, LYS31, TRP32 e HIS34; sul lato recettore, LEU27, PHE30, ASP32 e SER49, coerentemente con quanto atteso per l'interfaccia TGF-beta / recettore. Ogni riga del file `*_surface_patch.csv` rappresenta un punto della superficie molecolare, con coordinate cartesiane (x, y, z), normale (nx, ny, nz), densità elettronica approssimata e riferimento al residuo di appartenenza – formato pensato per essere direttamente compatibile con il successivo step di espansione in polinomi di Zernike (Task 2).

Il confronto tra i due pannelli è illustrativo delle differenze quantitative già riportate in Tabella 1 e nella Sezione 5. La patch di 1KTZ occupa un volume relativamente compatto (estensione massima di circa $28 \times 24 \times 19$ Å tra ligando e recettore) ed è composta da circa 12 500–12 900 punti di superficie per lato, coerentemente con un'interfaccia piccola (9 e 13 residui). La patch di 2OZA, al contrario, copre un'area sensibilmente più estesa e presenta ~87 700 punti sul ligando e ~86 400 sul recettore – circa sette volte più densa – riflettendo un'interfaccia con quasi otto volte più residui coinvolti (79 e 70) e una struttura complessiva di dimensioni maggiori (1493

Table 1: Conteggi della Task 1 per il complesso 1KTZ.

Grandezza	Valore
Atomi totali	1493
Atomi ligando (catena A)	653
Atomi recettore (catena B)	840
Residui di interfaccia – ligando	9
Residui di interfaccia – recettore	13
Contatti residuo–residuo	30
Atomi patch ligando	90
Atomi patch recettore	101
Punti superficie completa – ligando	84 247
Punti superficie completa – recettore	98 215
Punti superficie filtrata (interfaccia) – ligando	12 528
Punti superficie filtrata (interfaccia) – recettore	12 875

atomi totali per 1KTZ contro 5440 per 2OZA). Questo conferma visivamente che il numero di punti di superficie prodotti dalla pipeline scala con l'estensione reale dell'interfaccia biologica, e non è un artefatto dei parametri di campionamento (fissi per tutti i complessi).

5.2 Statistiche aggregate sui complessi dimerici

La Tabella 2 riassume la distribuzione delle grandezze principali calcolate dalla pipeline sui 120 complessi dimerici selezionati per il progetto. Le dimensioni dell'interfaccia (numero di residui e di contatti) e delle patch di superficie variano in modo considerevole anche all'interno di questo sottoinsieme, riflettendo l'eterogeneità dei sistemi dimerici presenti nel benchmark.

Table 2: Statistiche descrittive sui complessi dimerici selezionati per il progetto.

Grandezza	Media	Dev. std.	Mediana	Massimo
Atomi totali	5268	3948	4370	24 125
Residui interfaccia – ligando	25.2	9.7	24	79
Residui interfaccia – recettore	22.7	9.7	21	70
Contatti residuo–residuo	62.7	24.0	61	209
Atomi patch – ligando	205.7	77.1	200	639
Atomi patch – recettore	185.1	79.9	169	579
Punti superficie (interfaccia) – ligando	32 894	19 572	26 551	109 741
Punti superficie (interfaccia) – recettore	27 298	14 236	23 162	99 169

La Figura 2 mostra la distribuzione del numero di residui di interfaccia (per ligando e recettore) e del numero di contatti residuo–residuo sui complessi dimerici: la maggior parte delle interfacce coinvolge tra 15 e 35 residui per lato, con una coda di complessi ad interfaccia estesa (fino a ~80 residui), coerente con la varietà di sistemi anticorpo–antigene ed enzima–inibitore presenti nel sottoinsieme.

6 Task 2 – Mappatura della complementarità tramite descrittori di Zernike

Questa sezione è basata sulla lettura diretta dello script di produzione `run_task2_parallel.py`.

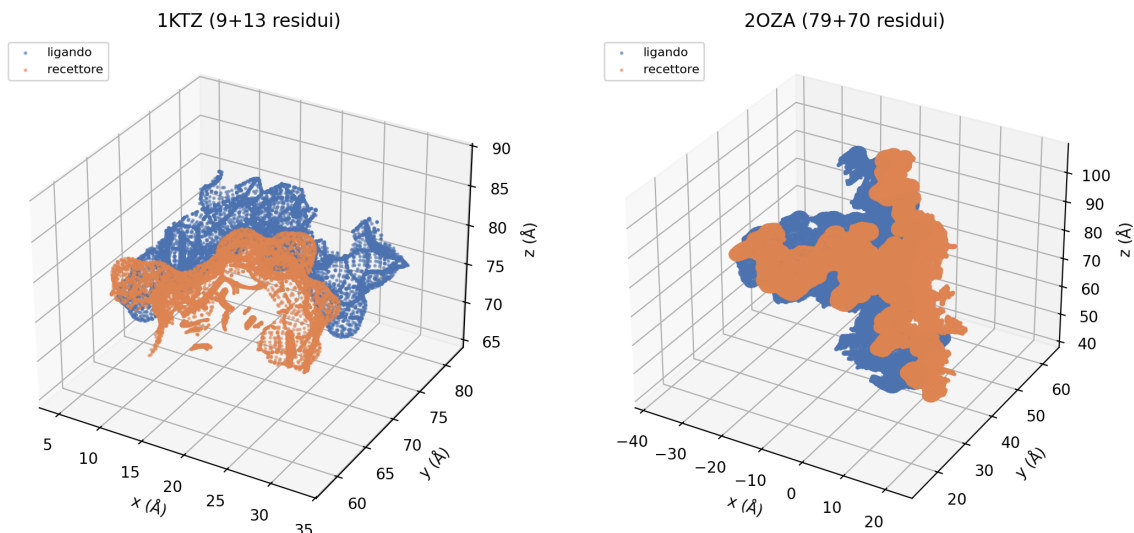


Figure 1: Visualizzazione 3D delle patch di superficie estratte per due complessi dimerici agli estremi opposti della distribuzione (assi in Å; in blu i punti della patch del ligando, in arancione quelli della patch del recettore). *Sinistra*: 1KTZ, l'interfaccia più piccola tra i dimeri selezionati (9 residui sul ligando, 13 sul recettore, 30 contatti residuo-residuo). *Destra*: 2OZA, l'interfaccia più estesa (79 residui sul ligando, 70 sul recettore, 209 contatti).

6.1 Nota metodologica: natura locale dei descrittori

La funzione utilizzata, `zepyros.get_zernike(coords, radius, center_index, order, verso)`, calcola il descrittore sulla patch di punti compresa entro un raggio fissato ($RADIUS = 6.0 \text{ Å}$) attorno a un singolo punto della superficie – non un vettore globale a dimensione fissa per l'intera proteina isolata. Questa firma è compatibile con il metodo di espansione locale in polinomi di Zernike descritto in Milanetti et al. (2020), lo stesso lavoro citato nella proposta di progetto.

Punto da verificare prima della consegna: se `zepyros` implementi effettivamente un'espansione 2D locale (patch proiettata su un piano di best-fit, come nel metodo di riferimento) oppure una base 3D. La firma della funzione è coerente con la prima ipotesi; la conferma va cercata nella documentazione del pacchetto o nel suo codice sorgente prima di caratterizzare questa scelta nel testo finale.

6.2 Algoritmo implementato

Per ciascun complesso, a partire dalle patch di superficie prodotte dalla Task 1 (`ligand_surface_patch.csv`, `receptor_surface_patch.csv`):

1. i punti vengono sottocampionati con passo $STRIDE = 20$ (indici 0, 20, 40, ...);
2. per ciascun punto campionato del ligando si calcola il descrittore di Zernike locale (ordine $ORDER = 20$, raggio 6.0 Å), con orientazione $verso = +1$; per ciascun punto campionato del recettore, lo stesso con $verso = -1$ (per mappare la corrispondenza concavità/convessità tra le due superfici);
3. si costruisce la matrice delle distanze euclidee *tra tutti* i descrittori del ligando e *tutti* i descrittori del recettore (`scipy.spatial.distance.cdist`) – si tratta quindi di un matching **globale nello spazio dei descrittori di forma**, non di un confronto ristretto alla coppia spazialmente più vicina;
4. per ogni punto del ligando si trattiene la distanza minima rispetto a tutti i punti del

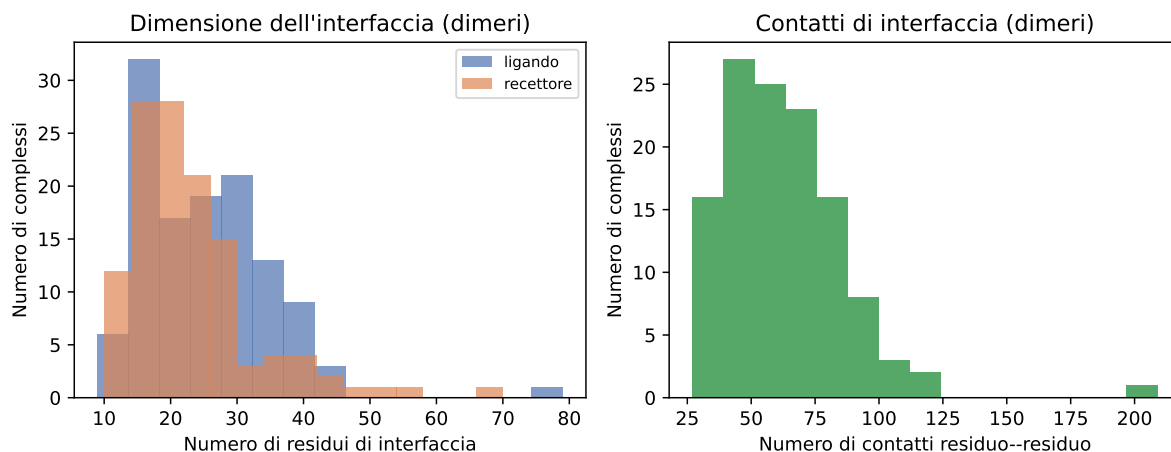


Figure 2: Distribuzione del numero di residui di interfaccia (sinistra) e del numero di contatti residuo-residuo (destra) sui complessi dimerici selezionati.

recettore (e viceversa per il recettore), definendo lo score `complementarity_score = - min(distanza)`: valori meno negativi indicano maggiore complementarità di forma;

5. i due versi (ligando→recettore e recettore→ligando) vengono uniti in un unico file, in modo che lo score complessivo non dipenda da quale dei due partner sia stato usato come riferimento.

L'output per complesso è `outputs/task2/{cartella}/zernike_complementarity_symmetric.csv`, con colonne `source` (ligand/receptor), `center_index`, coordinate `x,y,z` del centro, `zernike_distance` e `complementarity_score`.

6.3 Esecuzione su HPC

Il calcolo $-O(n_{\text{lig}} \times n_{\text{rec}})$ nel numero di punti campionati, per ciascuno dei quali si valuta un'espansione di ordine 20 – è stato eseguito su CINECA Leonardo con parallelizzazione a livello di complesso (`multiprocessing.Pool`, un processo per complesso, numero di core sincronizzato con `SLURM_CPUS_PER_TASK`). Alcuni complessi non hanno completato il calcolo entro i limiti di tempo delle code assegnate e sono stati esclusi dalle fasi successive: questa è la ragione della differenza tra il numero di dimeri selezionati nella Task 1 e il numero di complessi effettivamente utilizzati nelle Task 3 e 4.

6.4 Punti aperti

Nessuno script tra quelli esaminati esegue una validazione dello score di complementarità contro una *contact mask* nota (ad es. tramite curve ROC o Precision-Recall): se una validazione di questo tipo è stata effettivamente condotta in uno script separato, va aggiunta qui con riferimento al file corrispondente; altrimenti non va inclusa nel report.

7 Task 3 – Costruzione dei piani di complementarità 2D

[Sezione redatta a partire dalla documentazione descrittiva degli script; da verificare rileggendo il codice sorgente.]

7.1 Proiezione PCA sul piano medio di interfaccia

Per ciascun complesso viene caricata la nuvola di punti 3D dell'interfaccia prodotta dalla Task 2 e le si applica un'analisi delle componenti principali. Dopo centratura sul baricentro geometrico, si diagonalizza la matrice di covarianza 3×3

$$C = \frac{1}{N} X'^T X',$$

ottenendo gli autovettori $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ ordinati per autovalore decrescente. I primi due definiscono il **piano medio di best-fit** dell'interfaccia, su cui ogni punto viene proiettato tramite

$$x_{\text{proj}} = \mathbf{x}' \cdot \mathbf{u}_1, \quad y_{\text{proj}} = \mathbf{x}' \cdot \mathbf{u}_2,$$

mentre la coordinata lungo $\mathbf{u}_3$ misura la deviazione di ciascun punto dal piano, cioè la rugosità locale dell'interfaccia. La varianza spiegata da $\mathbf{u}_1 + \mathbf{u}_2$ quantifica il grado di planarità della patch. I risultati sono raccolti in `summary_geometrico_pca_task3.csv`.

7.2 Filtro energetico di Lennard-Jones

Rispetto alla traccia, il progetto introduce un passaggio aggiuntivo di pesatura fisica dei punti di interfaccia. Per ciascun punto si calcola la distanza reale dall'atomo più vicino del *partner opposto* (logica cross-interfaccia, implementata con due KD-Tree `scipy` indipendenti: punti del ligando contro atomi del recettore e viceversa). A questa distanza si applica una funzione di peso ispirata al potenziale di Lennard-Jones ($\sigma = 3.5 \text{ \AA}$), nulla per $r < 2.0 \text{ \AA}$ (regime repulsivo) e per $r > 6.0 \text{ \AA}$ (interazione trascurabile), e compresa tra 0 e 1 nella buca attrattiva di van der Waals.

Da qui si ottiene una complementarità geometrica media pesata energeticamente per ciascun complesso, che viene correlata con l'affinità sperimentale. Questo fornisce una **validazione analitica pre-deep-learning**: se il punteggio fisico-geometrico correla già di per sé con K_D , i dati forniti alla CNN contengono segnale e non solo rumore.

7.3 Discretizzazione su griglia 32×32 e canali

Le coordinate proiettate continue vengono interpolate su una griglia regolare 32×32 tramite `scipy.interpolate.griddata` (metodo `linear` con fallback `nearest`), scelta preferita rispetto alle RBF per evitare matrici singolari in presenza di punti densi o collineari. Si generano tre canali:

Canale 1 – Shape complementarity: modulo del *complementarity score* di Zernike.

Canale 2 – Potenziale elettrostatico: carica formale netta del residuo a pH fisiologico (7.4).

Canale 3 – Idrofobicità: valore della scala idropatica di Kyte-Doolittle del residuo di appartenenza, ricavato dalla colonna `residue_name` dei file di interfaccia della Task 1.

7.4 Standardizzazione Z-score globale

Una prima versione della pipeline normalizzava ogni mappa indipendentemente con una trasformazione Min-Max nel range $[0, 1]$. Questa scelta si è rivelata dannosa: azzerando le magnitudo assolute, un'interfaccia debolmente idrofobica e una fortemente idrofobica venivano mappate sullo stesso valore massimo, cancellando proprio l'informazione energetica rilevante per l'affinità. È stata quindi adottata una **standardizzazione Z-score globale**, calcolata canale per canale su tutti i pixel dell'intero dataset:

$$X_{\text{norm}}^{(c)} = \frac{X^{(c)} - \mu_s^{(c)}}{\sigma_s^{(c)}}.$$

L'output è un tensore `.npy` di forma $(3, 32, 32)$ per complesso, salvato in `outputs/task3/interface_maps/`, insieme a un'anteprima grafica a tre pannelli.

8 Task 4 – Predizione dell’affinità tramite CNN

[Sezione redatta a partire dalla documentazione descrittiva degli script; da verificare rileggendo il codice sorgente.]

8.1 Target

Il target di regressione è il logaritmo negativo della costante di dissociazione,

$$pK_D = -\log_{10} K_D,$$

scelta standard che linearizza un range di valori distribuito su diversi ordini di grandezza e lo rende proporzionale all’energia libera di legame.

8.2 Architettura InterfaceCNN

La rete (PyTorch, `models.py`) elabora tensori (3,32,32) tramite tre blocchi convoluzionali a contrazione spaziale progressiva ed espansione dei canali latenti. Ogni blocco applica `Conv2d` (3×3 , padding 1) $\rightarrow$ `BatchNorm2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d`(2):

$$(3, 32, 32) \rightarrow (16, 16, 16) \rightarrow (32, 8, 8) \rightarrow (64, 4, 4)$$

Segue il blocco di regressione: `Flatten` (1024) $\rightarrow$ `Linear`(1024, 128) $\rightarrow$ `Dropout`($p = 0.3$) $\rightarrow$ `Linear`(128, 1), con uscita lineare non vincolata. La scelta di una rete custom, anziché di architetture pre-addestrate tipo ResNet o VGG, è motivata dal *mismatch* tra immagini naturali e griglie biofisiche 32×32 a canali non-RGB.

8.3 Protocollo di addestramento

- **Loss:** errore quadratico medio (MSE), appropriato per regressione continua.
- **Ottimizzatore:** Adam, learning rate iniziale $\eta = 10^{-3}$, con scheduler `ReduceLROnPlateau` (factor 0.5, patience 7).
- **Validazione:** 5-fold cross-validation con raccolta delle predizioni *out-of-fold* (OOF). Le metriche finali sono calcolate una sola volta sul vettore OOF aggregato, evitando la dipendenza dalla fortuna di uno split fisso – criticità rilevante data la dimensione ridotta del dataset.
- **Data augmentation:** riflessioni orizzontali/verticali e rotazioni ortogonali discrete (90° , 180° , 270°) applicate a ogni epoca. La giustificazione è fisica: l’affinità è uno scalare termodinamico invariante per rotazione, mentre l’orientamento della griglia dipende da assi PCA arbitrari; l’augmentation inietta quindi un vincolo noto a priori anziché un’arbitrarietà.
- **Hardware:** accelerazione su Apple Silicon tramite backend PyTorch MPS.

8.4 Risultati

La Tabella 3 riporta l’evoluzione delle metriche attraverso le tre configurazioni testate, valutate sulle predizioni OOF.

Il salto dalla configurazione 1 alla 2 è attribuibile congiuntamente a due modifiche (schema di validazione e schema di normalizzazione), che non sono state variate indipendentemente: l’effetto delle due non è quindi separabile sulla base di questi dati soltanto. L’ulteriore guadagno della configurazione 3 è modesto in RMSE e più marcato nella correlazione di rango.

Table 3: Metriche di predizione dell’affinità (predizioni out-of-fold).

Configurazione	RMSE	Pearson R	Spearman ρ
1. Split fisso + Min-Max locale	2.2133	0.1139	0.1746
2. 5-fold CV + Z-score globale	1.8789	0.5071	0.4968
3. + Data augmentation e LR scheduler	1.8441	0.5333	0.5396

9 Discussione e limiti

Dimensione del campione. Il limite dominante è la numerosità: circa un centinaio di complessi. Con $n \approx 120$, un coefficiente di Pearson di 0.53 ha un intervallo di confidenza al 95% indicativamente compreso tra 0.39 e 0.65: il segnale è reale e statisticamente distinguibile da zero, ma la stima puntuale va presentata con la sua incertezza, e differenze di qualche centesimo tra configurazioni non sono significative.

Standardizzazione e leakage. Le statistiche μ_s e σ_s della standardizzazione Z-score sono calcolate sull’intero dataset prima della cross-validation, dunque includendo i fold di validazione. Si tratta di una forma lieve di *data leakage*: le statistiche sono non supervisionate e l’impatto atteso è piccolo, ma la procedura rigorosa richiederebbe di ricalcolarle sul solo training set di ciascun fold. Vale la pena dichiararlo, ed eventualmente verificarne l’effetto con un controllo.

Confronti con baseline. La traccia richiede il confronto con modelli di geometric deep learning di riferimento e con descrittori Zernike puri. Le configurazioni 1–3 sono ablazioni interne della stessa pipeline, non confronti con baseline esterne. L’analisi di correlazione di Lennard-Jones (Sezione 7) costituisce di fatto una baseline “Zernike-only”, e in quanto tale andrebbe presentata esplicitamente nella tabella comparativa; il confronto con modelli di geometric DL resta invece scoperto.

Scostamento dalla traccia. L’uso di descrittori 3D anziché 2D (Sezione 6) va dichiarato apertamente, con la motivazione e il relativo costo computazionale. Presentarlo come scelta consapevole e documentata è preferibile a lasciarlo implicito.

Interpretabilità. La traccia richiede una mappatura interpretabile tra feature di interfaccia e affinità. L’estrazione di mappe di salienza tramite gradienti è la via naturale e va inclusa solo se effettivamente calcolata.

10 Sviluppi futuri

La direzione naturale di estensione è il passaggio a una predizione *blind*, in cui il modello riceva le superfici complete delle due proteine isolate senza conoscere a priori la regione di legame. Questo trasformerebbe il lavoro da esercizio di regressione supervisionata su interfacce note a strumento di screening virtuale applicabile a coppie di proteine mai co-cristallizzate, ad esempio a partire da strutture predette. Va notato che in questo scenario – e non in quello attuale – l’invarianza rotazionale intrinseca dei descrittori 3D globali diventa il vantaggio decisivo.